

JMAP Migration Throughput

Findings, forecast and open risks · Vandelay migration tool vs. Stalwart JMAP · 25 July 2026

Bottom line. The migration tool is not the bottleneck and no longer needs work for speed. Against a local server it moves **1,500 messages per second**. Against our remote Stalwart it moves **5–8 per second** — roughly **200× slower on identical software**. The remaining limit is how fast the remote server accepts messages, not how fast we send them.

What changed

The tool previously sent two network requests per message: one to upload the message body, one to file it. Servers cap how many requests one account may have in flight, so the number of requests — not bandwidth — set the speed limit.

It now groups messages into batches using a newer part of the JMAP standard (RFC 9404), so one request carries many messages. Measured effect:

Environment	Before	After	Change
Local server (no network delay)	13.6 msg/s	1,507 msg/s	110× faster
Remote server, like-for-like test	5.6 msg/s	7.2 msg/s	29% faster

The gap between those two rows is the whole story. The same change that makes the tool 110× faster locally buys only 29% remotely, because remotely we spend our time waiting for the server rather than issuing requests.

What to expect for a real migration

Forecast at **5–8 messages per second**, measured with realistically sized (50 KB) messages against the current remote server:

Mailbox size	Expected duration	If server were as fast as local
10,000 messages	21 min – 33 min	7 seconds
50,000 messages	1.7 h – 2.8 h	33 seconds
100,000 messages	3.5 h – 5.6 h	1 minute
1,000,000 messages	35 h – 56 h	11 minutes

These are single-account figures. Several accounts migrated in parallel would multiply throughput only if the server's limit is per account rather than global — which we have *not* yet tested. See open questions.

Where the remaining time goes

We separated the two halves of the work to find the constraint. Uploading message bodies is fast and scales with parallelism. Filing them into the mailbox does not:

Parallel workers	Body upload	Filing (Email/import)	End to end
4	17.7/s	8.3/s	5.7 msg/s
8	35.3/s	11.4/s	8.5 msg/s
16	22.8/s	13.7/s	8.5 msg/s

Filing is the bottleneck, and adding parallelism stops helping past 8 workers. On the local server the identical operation runs over a hundred times faster, so this is not something inherent to the software — it is specific to how the remote server is configured or hosted.

What the rejections mean — and do not mean

No mail is lost. When the server refuses a request for exceeding its concurrency limit, the tool recognises that specific refusal as temporary, waits, and retries it. In our production test runs every message arrived: `created=600`, `failed=0`.

What rejections actually cost is **time**, and they are a signal we are pushing harder than the server wants:

- Each rejection means a wasted round trip plus a backoff wait before retrying.
- Retries are capped (5 by default). If a message exhausted all retries it would be reported as failed and counted in the run summary — it would not fail silently.
- The tool is *convergent* by design: re-running it imports only what is missing. A partial or interrupted run is safe to simply repeat.

Practically: rejections are a tuning signal, not a data-integrity problem. We saw them appear above 8 parallel workers, which is exactly why 8 is our recommended setting.

Recommended settings

Setting	Value	Why
<code>--threads</code>	8	Measured best. 4 is slower (4.8 msg/s); 16 is slower still and starts triggering rejections.
<code>--progress</code>	on	Newly added. Shows completed/total, rate and estimated time remaining, so a multi-hour run is observable.

Server-side: where the real speedup is

Raising the server's concurrency limits from 4 to 16 and lifting the upload quota both worked and both helped. Neither will help further, because filing messages does not scale with concurrency. The 200x local-versus-remote gap on identical software points at the environment. In likely order:

- **Storage backend.** If the remote server stores mail on network or object storage while the local one uses a local disk, that alone could explain the gap.
- **Full-text indexing on import.** Indexing every message as it arrives is expensive; pausing it during a bulk migration and rebuilding afterwards is a common and safe tactic.
- **Disk-sync policy.** Forcing a flush to disk per message is a large per-message cost.

This is the single highest-value item on the list. Everything on the client side is already far faster than the server can absorb.

Honest limitations of these numbers

- **Server load varied during testing.** Baseline round-trip time to the remote server moved between 27 ms and 84 ms across the session, a 3× swing. Only measurements taken back-to-back are reliable; absolute figures from different hours are not comparable. The forecast range above reflects this.
- **Attachment-heavy mail is untested.** We tested 150-byte and 50 KB messages. Mailboxes with large attachments will behave differently and may become bandwidth-limited rather than server-limited.
- **The tool has more headroom than we have proven.** A stripped-down test harness reached 13.3 msg/s where the production tool reached 7.5 msg/s. We have not yet isolated whether that is server load drift or a real inefficiency in how the tool interleaves its two phases. Worth roughly 40% if it is real.
- **Multi-account parallelism is unmeasured.** If limits are per account, migrating several mailboxes concurrently is the cheapest available speedup. This has not been tested.
- **One test harness run lost 10 of 200 messages.** This occurred in the simplified measurement harness, which deliberately omits the retry logic; the production tool reported zero failures in every run. It should still be confirmed rather than assumed.
- **IMAP was not evaluated.** IMAP bypasses the JMAP upload path entirely and might be faster, but our JMAP credentials were rejected by the IMAP service and we stopped rather than risk triggering account lockout.

Approaches we tested and rejected

- **WebSocket transport.** 7× faster for trivial requests, but flat at ~3 msg/s for actual imports at every configuration tried — the server appears to process one request at a time per socket. Not worth building on.
- **More parallel connections.** The server counts in-flight requests per account, not per connection. Opening more connections changes nothing; we verified this directly.
- **Very large batches.** Beyond roughly 100 messages per request, throughput falls again.

Recommended next steps

1. **Investigate the remote server's storage and indexing configuration.** Highest value by a wide margin; everything else is incremental.
2. **Test migrating several accounts concurrently** to establish whether limits are per account. Cheap to test, potentially a large multiplier.
3. **Run one full-size trial migration** on a representative mailbox before committing to a cutover window, so the forecast is validated on real data rather than synthetic messages.
4. **Confirm IMAP access** so that path can be benchmarked as an alternative.

committed alongside the tool so any figure here can be reproduced.